



UNIVERSIDAD NACIONAL DEL NORDESTE
Facultad de Ciencias Exactas y Naturales y Agrimensura

Licenciatura en Sistemas de Información |
Ingeniería de Software 2

INTEGRANTES:

- Pinat, Juan Matias – 39.190.409
- Lugo, Renata Aylén – 44.572.954

Primera Entrega - Trabajo de campo - Grupo 23.

2026

ÍNDICE

1. Introducción

- 1.1 Breve estado del arte
- 1.2 Objetivos
- 1.3 Fundamentación

2. Metodología

- 2.1 Ciclo de vida
- 2.2 Arquitectura de software
- 2.3 Planificación
 - 2.3.1 Sprints
 - 2.3.2 Historias de usuario
- 2.4 Estimación de costos
 - 2.4.1 Clasificación del proyecto
 - 2.4.2 Estimación del tamaño del sistema
 - 2.4.3 Resultados del modelo COCOMO Básico
 - 2.4.4 Ajuste al cronograma real
 - 2.4.5 Costo de herramientas
 - 2.4.6 Resumen general de costos
 - 2.4.7 Conclusión
- 2.5 Especificación de requerimientos de software
 - 2.5.1 Introducción
 - 2.5.1.1 Propósito
 - 2.5.1.2 Alcance
 - 2.5.1.3 Definiciones, acrónimos y abreviaturas
 - 2.5.2 Descripción general
 - 2.5.2.1 Perspectiva del producto
 - 2.5.2.2 Funcionalidad del producto
 - 2.5.2.3 Características de los usuarios
 - 2.5.2.4 Restricciones
 - 2.5.2.5 Suposiciones y dependencias
 - 2.5.3 Requisitos específicos
 - 2.5.3.1 Requisitos funcionales
 - 2.5.3.2 Requisitos no funcionales
 - 2.5.3.3 Requisitos organizacionales
 - 2.5.3.4 Requisitos externos
- 2.6 Descripción global
 - 2.6.1 Perspectiva del producto
 - 2.6.2 Funcionalidad del producto
 - 2.6.3 Características del usuario
 - 2.6.4 Restricciones
 - 2.6.5 Suposiciones y dependencias
- 2.7 Plan de riesgos
 - 2.7.1 Escala de evaluación

2.8 Modelado del sistema

2.8.1 Diagrama de casos de uso

2.8.2 Conversaciones

2.8.3 Diagramas de secuencia

2.8.4 Contrato de operaciones críticas

2.8.5 Diagrama entidad-relación

3. Herramientas y/o lenguajes de programación

4. Resultados

4.1 Estado actual del módulo de reparaciones

5. Conclusión

5.1 Bibliografía

ÍNDICE DE FIGURAS

Figura Nro 1 - Diagrama de la arquitectura modelo-vista-controlador

Figura Nro 2 - Trello

Figura Nro 3 - Diagrama de Gantt

Figura Nro 4 - Caso de uso: visitante

Figura Nro 5 - Caso de uso: cliente

Figura Nro 6 - Caso de uso: administrador

Figura Nro 7 - Diagrama de secuencia: cargar al carrito

Figura Nro 8 - Diagrama de secuencia: realizar compra

Figura Nro 9 - Diagrama de secuencia: solicitar reparación

Figura Nro 10 - Diagrama entidad-relación

Figura Nro 11 - Catálogo de productos (como visitante)

Figura Nro 12 - Catálogo de productos (como cliente)

Figura Nro 13 - Catálogo de productos (como administrador)

Figura Nro 14 - Carga de productos al carrito

Figura Nro 15 - Carrito cargado con productos

Figura Nro 16 - Compra finalizada

ÍNDICE DE TABLAS

Tabla Nro 1 - Sprints

Tabla Nro 2 - Historia de usuario 1

Tabla Nro 3 - Historia de usuario 2

Tabla Nro 4 - Historia de usuario 3

Tabla Nro 5 - Historia de usuario 4

Tabla Nro 6 - Historia de usuario 5

Tabla Nro 7 - Historia de usuario 6

Tabla Nro 8 - Historia de usuario 7

Tabla Nro 9 - Historia de usuario 8

Tabla Nro 10 - Estimación del tamaño del problema

Tabla Nro 11 - Capacidad del cronograma

Tabla Nro 12 - Costo del proyecto

Tabla Nro 13 - Costo de herramientas

Tabla Nro 14 - Resumen general de costos

Tabla Nro 15 - Características de los usuarios

Tabla Nro 16 - Requisitos funcionales

Tabla Nro 17 - Requisitos no funcionales

Tabla Nro 18 - Requisitos organizacionales

Tabla Nro 19 - Requisitos externos

Tabla Nro 20 - Característica de los usuarios

Tabla Nro 21 - Probabilidad de ocurrencia

Tabla Nro 22 - Niveles de efecto

Tabla Nro 23 - Análisis de riesgo

Tabla Nro 24 - Planificación de riesgos

Tabla Nro 25 - Conversación "Realizar compra"

Tabla Nro 26 - Conversación "Agregar producto"

Tabla Nro 27 - Conversación "Solicitar reparación"

Tabla Nro 28 - Contrato de operación 1

Tabla Nro 29 - Contrato de operación 2

1. INTRODUCCIÓN

1.1 Breve estado del arte

Actualmente, muchas tiendas de tecnología utilizan plataformas web para centralizar la gestión de ventas, productos y atención al cliente. Este tipo de sistemas permite mejorar la organización del negocio, agilizar los procesos de compra y brindar una mejor experiencia a los usuarios.

En el caso de NorTech, se buscó desarrollar una solución orientada a un ecommerce simple pero funcional, tomando como referencia características presentes en tiendas online modernas, como autenticación de usuarios, catálogo de productos, carrito de compras y gestión de ventas. Además, se incorporó de manera conceptual un módulo de solicitudes de reparación para ampliar el alcance del sistema.

Para el desarrollo del proyecto se utilizaron tecnologías actuales ampliamente utilizadas en aplicaciones web, como React para el frontend, Node.js para el backend y PostgreSQL como sistema gestor de base de datos. La combinación de estas herramientas permitió construir una aplicación modular, escalable y adaptada a los requerimientos definidos para el trabajo de campo.

1.2 Objetivos

El proyecto tiene como propósito desarrollar un software para automatizar el sistema de ventas de un local, especializado en la comercialización y reparación de teléfonos celulares, será capaz de cubrir todos los requerimientos planteados por sus usuarios y administradores.

La interfaz del software será intuitiva y accesible, pensada para facilitar la navegación tanto del equipo administrativo como de los clientes que interactúen con la plataforma.

El sistema contará con las siguientes funcionalidades:

- Consultas y registros de las ventas por productos.
- Consultas, ingreso y registro de reclamos, y por cada reclamo, generará una nota de crédito.
- El sistema debe realizar ventas a los clientes finales.
- El sistema también agilizará los procesos de ventas, tanto en el estado de recibir los productos como en el pago del mismo.
- Almacenará todos los datos respectivos a la venta, entre ellos se puede detallar la fecha de la transacción, los productos vendidos, el valor total, etc.

- Permitirá realizar y consultar las estadísticas de ventas mensuales.
- Módulo de solicitud y seguimiento para servicios de reparación de celulares, permitiendo al cliente registrar su equipo, describir el problema y consultar el estado de su reparación en tiempo real.

1.3 Fundamentación

La motivación principal detrás de este proyecto surge de la necesidad de digitalizar y optimizar la operación de la tienda "NorTech", para gestionar tanto la venta como la reparación de teléfonos celulares utilizando una herramienta tecnológica adecuada. La ausencia de un sistema integrado genera demoras, errores en el registro de información y una experiencia de cliente poco satisfactoria. El desarrollo de esta plataforma permitirá automatizar los procesos cotidianos, centralizar la información de productos, clientes, ventas y órdenes de reparación, y mejorar sustancialmente la organización interna del negocio.

2. METODOLOGÍA

2.1 Ciclo de vida

El desarrollo se basa en el ciclo de vida del software, el cual comprende un conjunto de etapas que permiten planificar, construir, implementar y mantener un sistema informático de manera organizada. Este ciclo incluye la planificación, donde se definen los objetivos del proyecto; el análisis de requisitos, en el que se identifican las necesidades del usuario, como la visualización de productos, el carrito de compras y los medios de pago; y el diseño, donde se establece la arquitectura del sistema, la base de datos y la interfaz de usuario. Posteriormente, en la fase de desarrollo se implementan las funcionalidades definidas, las cuales son sometidas a pruebas para verificar su correcto funcionamiento. Una vez validadas, se realiza la implementación del sistema en un entorno real para su uso por parte de los usuarios. Finalmente, el software entra en una etapa de mantenimiento continuo, donde se realizan mejoras, correcciones y actualizaciones en función de la evolución del negocio y las necesidades del mercado.

Para gestionar este ciclo de vida, se adopta la metodología Scrum, un marco de trabajo ágil que organiza el desarrollo en ciclos cortos denominados sprints. En cada sprint se planifican, desarrollan y prueban funcionalidades completas del sistema, permitiendo avanzar de manera progresiva mediante entregas parciales que aportan valor al producto. Se eligió Scrum porque permite organizar mejor las tareas del equipo y adaptar funcionalidades durante el desarrollo sin afectar el avance general del proyecto, donde es

necesario adaptarse constantemente a cambios como la incorporación de nuevos productos, variaciones en los precios o mejoras en la experiencia del usuario. A diferencia de enfoques tradicionales, Scrum permite una mayor flexibilidad, facilita la incorporación de cambios durante el proceso y promueve la entrega continua de resultados, reduciendo riesgos y mejorando la calidad del producto final.

2.2 Arquitectura de software

A la hora de diseñar el sistema se consideraron las siguientes arquitecturas para llevar a cabo el desarrollo del proyecto NorTech:

- **Arquitectura en capas:** Divide la aplicación en bloques bien diferenciados, cada uno con una responsabilidad definida. Favorece la organización lógica del código, facilitando el mantenimiento, la escalabilidad y la reutilización de componentes a lo largo del ciclo de vida del sistema.
- **Modelo Vista Presentador:** Patrón orientado a separar la lógica de negocio de la interfaz gráfica. Mejora la testeabilidad y el mantenimiento de la UI al delegar la lógica al presentador, quien actúa como intermediario entre la vista y el modelo de datos.

Luego de analizar las alternativas disponibles, se optó por la arquitectura **Modelo-Vista-Controlador (MVC)**, por ser la que mejor se adapta a las necesidades de una tienda online de celulares con módulo de reparaciones, ya que permite una separación clara entre la lógica de negocio, la presentación de la información y el control del flujo de la aplicación. La arquitectura MVC también facilitó dividir el trabajo entre frontend y backend durante el desarrollo práctico del sistema.

a) Separación de responsabilidades:

- **Modelo:** se encargará de la gestión de datos y las reglas de negocio, incluyendo el catálogo de productos, los pedidos de compra, los clientes y las órdenes de reparación.
- **Vista:** representará la interfaz visual que será presentada a los distintos usuarios, tanto clientes que navegan la tienda como administradores que gestionan el inventario y las reparaciones.
- **Controlador:** gestionará la interacción del usuario, la validación de datos y el flujo de navegación, actuando como intermediario entre el modelo y la vista en operaciones como el procesamiento de pedidos, el registro de reclamos o el

seguimiento del estado de una reparación.

b) Escalabilidad y mantenibilidad: esta arquitectura permite incorporar nuevas funcionalidades en el futuro —como nuevos métodos de pago, categorías de productos o tipos de servicio técnico— sin comprometer la lógica central ni la interfaz existente. La independencia entre sus tres componentes facilita un mantenimiento modular y ordenado.

c) Facilitación de pruebas: es posible realizar pruebas unitarias sobre el controlador y el modelo de forma independiente, sin necesidad de involucrar la interfaz gráfica, lo cual agiliza los ciclos de desarrollo y la detección temprana de errores.

La elección de la arquitectura Modelo-Vista-Controlador proporciona una base sólida para estructurar el proyecto NorTech, gracias a su modularización, escalabilidad y facilidad de mantenimiento. Esta arquitectura se alinea con las tecnologías seleccionadas para el desarrollo del sistema, permitiendo construir una plataforma robusta, intuitiva y flexible que soporte tanto las operaciones de comercio electrónico como la gestión integral del servicio de reparación de dispositivos móviles.

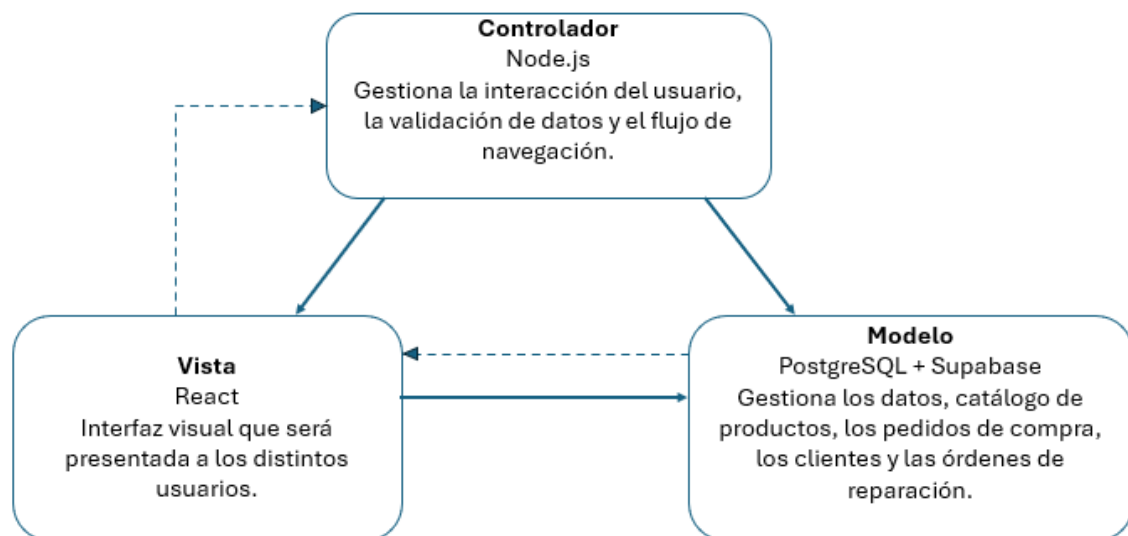


Figura Nro 1 - Diagrama de la arquitectura modelo-vista controlador

2.3 Planificación

En nuestro proyecto, hemos optado por utilizar Trello como nuestra herramienta principal de gestión de proyectos.

Trello es una plataforma versátil y poderosa que nos permite planificar, rastrear y colaborar en todas las etapas del ciclo de vida del proyecto de manera efectiva.

Facilita la colaboración entre los miembros del equipo al proporcionar una plataforma

centralizada para la comunicación y el intercambio de información. Los comentarios en tiempo real, las actualizaciones de estado y la integración con otras herramientas de colaboración que nos permiten trabajar de manera eficiente y mantenernos sincronizados en todo momento.



Figura Nro 2 - Trello

2.3.1 Sprints

Sprint	Duración	Objetivo	Tareas	Responsable de cada tarea
Sprint 1: Planificación inicial	30/03 - 02/04	Establecer las bases del proyecto y organizar el entorno de trabajo	-Reunión inicial - Debate idea del proyecto - Creación del repositorio en GitHub y carpeta en Drive - Realización del ciclo de vida -Debate de las funcionalidades y tipo de arquitectura	Ambos participantes participaron en la toma de decisiones para la planificación inicial.
Sprint 2: Desarrollo de diagramas	11/04 - 19/04	Modelar el sistema a través de diagramas UML, conversaciones y contratos de operación	- Casos de uso (3) - Conversaciones(3) - Diagrama de secuencia(3) - Contrato de operaciones críticas(2) -Diagrama entidad-relación	La cantidad que se solicitó con respecto a cada tipo de diagrama fue dividida de forma equitativa para los dos integrantes
Sprint 3: Plan de riesgos	10/04 - 11/04	Identificar, analizar y planificar la respuesta ante los riesgos del proyecto	- Análisis de riesgo - Planificación de riesgos	Ambos participantes a través de una videoconferencia planificaron y analizaron los riesgos del proyecto

Sprint 4: Desarrollo de la aplicación	11/04 - 20/04	Implementar las funcionalidades principales del sistema	- Actualizar la base de datos - Desarrollo del carrito de compras - Desarrollo del servicio de reparación de celulares	Ambos integrantes implementaron las funcionalidades del proyecto a través de videoconferencias y de forma presencial
Sprint 5: Desarrollo de pruebas	20/04 - 29/04	Validar el funcionamiento del sistema mediante pruebas	- Ejecución de la primera funcionalidad	Los integrantes a lo largo de la primera entrega fueron ejecutando la funcionalidad, para ir comprobando el funcionamiento del mismo.

Tabla Nro 1 - Sprints

teamgantt
Created with Free Edition

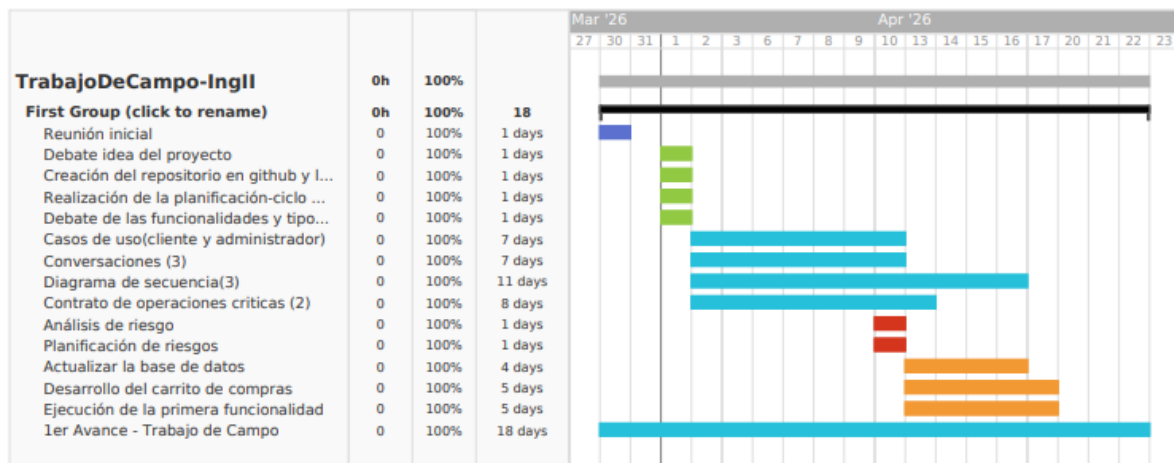


Figura Nro 3 - Diagrama de Gantt

2.3.2 Historias de usuario

Número:	1
Usuario:	Visitante
Nombre Historia:	Visualizar catálogo de celulares
Prioridad de Negocio:	Alta
Riesgo en Desarrollo:	Bajo
Puntos de Historia:	1
Programador Responsable:	Pinat Juan Matias y Lugo Renata Aylen
Descripción:	Como visitante quiero visualizar el catálogo de celulares disponibles para conocer los productos antes de realizar una compra.

Tabla Nro 2 - Historia de usuario 1

Número:	2
Usuario:	Cliente
Nombre Historia:	Agregar productos al carrito
Prioridad de Negocio:	Alta
Riesgo en Desarrollo:	Medio
Puntos de Historia:	2
Programador Responsable:	Pinat Juan Matias y Lugo Renata Aylen
Descripción:	Como cliente quiero agregar productos al carrito para seleccionar los celulares que deseo comprar.

Tabla Nro 3 - Historia de usuario 2

Número:	3
Usuario:	Cliente
Nombre Historia:	Eliminar productos del carrito
Prioridad de Negocio:	Alta
Riesgo en Desarrollo:	Bajo
Puntos de Historia:	1
Programador Responsable:	Pinat Juan Matias y Lugo Renata Aylen
Descripción:	Como cliente quiero eliminar productos del carrito para modificar mi selección antes de confirmar la compra.

Tabla Nro 4 - Historia de usuario 3

Número:	4
Usuario:	Cliente
Nombre Historia:	Vaciar carrito
Prioridad de Negocio:	Media
Riesgo en Desarrollo:	Bajo
Puntos de Historia:	1
Programador Responsable:	Pinat Juan Matias y Lugo Renata Aylen
Descripción:	Como cliente quiero vaciar el carrito para eliminar todos los productos seleccionados rápidamente.

Tabla Nro 5 - Historia de usuario 4

Número:	5
Usuario:	Cliente
Nombre Historia:	Finalizar compra
Prioridad de Negocio:	Alta
Riesgo en Desarrollo:	Alto
Puntos de Historia:	2
Programador Responsable:	Pinat Juan Matias y Lugo Renata Aylen
Descripción:	Como cliente quiero confirmar la compra de los productos del carrito para generar una venta en el sistema.

Tabla Nro 6 - Historia de usuario 5

Número:	6
Usuario:	Cliente
Nombre Historia:	Visualizar resumen de compra
Prioridad de Negocio:	Alta
Riesgo en Desarrollo:	Medio
Puntos de Historia:	1
Programador Responsable:	Pinat Juan Matias y Lugo Renata Aylen
Descripción:	Como cliente quiero ver el detalle de los productos seleccionados y el total a pagar para verificar la compra antes de confirmarla.

Tabla Nro 7 - Historia de usuario 6

Número:	7
Usuario:	Sistema
Nombre Historia:	Validar stock de productos
Prioridad de Negocio:	Alta
Riesgo en Desarrollo:	Medio
Puntos de Historia:	2
Programador Responsable:	Pinat Juan Matias y Lugo Renata Aylen
Descripción:	Como sistema quiero validar la disponibilidad de stock antes de confirmar la compra para evitar ventas de productos no disponibles.

Tabla Nro 8 - Historia de usuario 7

Número:	8
Usuario:	Cliente
Nombre Historia:	Solicitar reparación (conceptual)
Prioridad de Negocio:	Media
Riesgo en Desarrollo:	Medio
Puntos de Historia:	1
Programador Responsable:	Pinat Juan Matias y Lugo Renata Aylen
Descripción:	Como cliente quiero solicitar la reparación de un celular para dejar registrada una solicitud en el sistema (modelada en el DER como funcionalidad futura).

Tabla Nro 9 - Historia de usuario 8

2.4 Estimación de costos

Para la estimación de costos del proyecto se aplicó el método COCOMO Básico (Constructive Cost Model), un modelo ampliamente utilizado en ingeniería de software que permite estimar el esfuerzo, el tiempo y el costo de desarrollo en función del tamaño del sistema medido en miles de líneas de código fuente (KLOC). Este método resulta apropiado para el contexto del proyecto, dado que se trata de un equipo reducido con roles bien definidos y un alcance académico delimitado.

2.4.1 Clasificación del proyecto

COCOMO Básico clasifica los proyectos en tres modos según su complejidad y características del equipo. Para este proyecto se seleccionó el modo orgánico, dado que el equipo es pequeño, los requisitos están bien definidos desde el inicio y el entorno de desarrollo resulta familiar para los integrantes.

Las fórmulas aplicadas para el modo orgánico son las siguientes:

- **Esfuerzo:** $E = 2.4 \times (\text{KLOC})^{1.05}$ (expresado en personas-mes)
- **Tiempo de desarrollo:** $T = 2.5 \times (E)^{0.38}$ (expresado en meses)
- **Personas requeridas:** $P = E / T$

2.4.2 Estimación del tamaño del sistema

Se estimó la cantidad de líneas de código por módulo, contemplando el desarrollo tanto del frontend en React como del backend en Node.js. El alcance fue ajustado en función del plazo real disponible de dos meses:

Módulo	Líneas de código estimadas
Autenticación y gestión de usuarios	400
Gestión de productos (CRUD)	450
Carrito de compras y proceso de compra	500
Gestión de ventas y reportes básicos	350
Módulo de solicitudes de reparación	500
Configuración de base de datos	300

(PostgreSQL / Tembo.io)	
Componentes generales de interfaz (React)	400
Total estimado:	2900 líneas

Tabla Nro 10 - Estimación del tamaño del problema

KLOC = 2.9

2.4.3 Resultados del modelo COCOMO Básico

Aplicando las fórmulas del modelo sobre el valor de KLOC obtenido:

Esfuerzo: $E = 2.4 \times (2.9)^{1.05} = 2.4 \times 3.09 \approx 7.41$ **personas-mes**

Tiempo de desarrollo teórico: $T = 2.5 \times (7.41)^{0.38} = 2.5 \times 2.28 \approx 5.7$ **meses**

Personas requeridas: $P = 7.41 / 5.7 \approx 1.3$ **persona**

2.4.4 Ajuste al cronograma real

El modelo COCOMO arroja un tiempo teórico de desarrollo de 5.7 meses. Sin embargo, el proyecto debe completarse en un plazo de 2 meses, con un equipo de 2 integrantes. A continuación se detalla la capacidad disponible:

Parámetro	Valor
Tiempo disponible	2 meses
Integrantes del equipo	2 personas
Capacidad disponible	4 personas-mes
Esfuerzo estimado por COCOMO	7.41 personas-mes
Porcentaje del alcance cubierto	Aproximadamente 54%

Tabla Nro 11 - Capacidad del cronograma

Dado que la capacidad real del equipo cubre aproximadamente el 54% del esfuerzo total estimado, el desarrollo se concentrará en las funcionalidades de mayor valor para el negocio: autenticación de usuarios, gestión de productos, proceso de compra y módulo de reparaciones en su versión básica. Funcionalidades secundarias como reportes avanzados y estadísticas de ventas quedan previstas para una segunda iteración del sistema.

Para calcular el costo del proyecto se tomó como referencia el valor de mercado de un desarrollador junior en Argentina al momento del desarrollo:

Concepto	Detalle	Costo estimado
Costo por persona-mes	Desarrollador junior	\$400.000
Capacidad real del equipo	2 personas × 2 meses = 4 personas-mes	-
Costo total del desarrollo		\$1.600.000

Tabla Nro 12 - Costo del proyecto

2.4.5 Costo de herramientas

Recursos	Observación	Costo estimado
GitHub	Plan gratuito	\$0
Visual Studio Code	Plan gratuito	\$0
Trello	Plan gratuito	\$0
Conexión a internet	2 personas x 2 meses	\$60.000
Total:		\$60.000

Tabla Nro 13 - Costo de herramientas

2.4.6 Resumen general de costos

Concepto	Costo estimado
Desarrollo	\$1.600.000
Herramientas	\$60.000
Costo total:	\$1.660.000

Tabla Nro 14 - Resumen general de costos

2.4.7 Conclusión

A partir de la aplicación del modelo COCOMO Básico en modo orgánico, se determinó que el desarrollo completo del sistema requeriría aproximadamente 7.41 personas-mes de esfuerzo en condiciones normales. Dado que el equipo dispone de un plazo de 2 meses con 2 integrantes, la capacidad efectiva disponible es de 4 personas-mes, equivalente al 54% del esfuerzo total estimado. En consecuencia, el desarrollo priorizará las funcionalidades esenciales del sistema, asegurando una entrega operativa dentro del cronograma académico establecido. El costo total estimado del proyecto asciende a \$1.660.000, considerando tanto los costos de mano de obra como los de infraestructura tecnológica.

2.5 Especificación de requerimientos de software

2.5.1 Introducción

2.5.1.1 Propósito

El presente documento tiene como propósito definir las especificaciones funcionales y no funcionales para el desarrollo de una plataforma web que permita gestionar la venta de teléfonos celulares y los servicios de reparación de dispositivos móviles de la empresa NorTech. El sistema será utilizado tanto por el administrador de la tienda como por los clientes registrados y no registrados que interactúen con la plataforma.

2.5.1.2 Alcance

Esta especificación de requisitos está dirigida a la empresa "NorTech", para el desarrollo del sistema "System NorTech", el cual permitirá la gestión de productos, clientes, ventas online y solicitudes de reparación de celulares, incluyendo el alta, baja y modificación de cada uno de estos registros.

2.5.1.3 Definiciones, acrónimos y abreviaturas

Jefe: Persona que se encuentra en el puesto superior de una jerarquía organizacional y que tiene las facultades necesarias para dirigir y coordinar a sus subordinados.

Cliente: Persona que, a cambio de un pago, recibe productos o servicios ofrecidos por la empresa.

Administrador: Persona a cargo de la dirección o coordinación de la plataforma, con acceso completo a la gestión de productos, clientes, ventas y reparaciones.

Software: Conjunto de programas y componentes lógicos que permiten la realización de tareas específicas en un sistema informático.

Sistema operativo: Software principal que gestiona los recursos de hardware de un equipo y provee servicios a los programas de aplicación.

Base de datos: Colección de información organizada que permite a un sistema informático almacenar, consultar y manipular datos de forma eficiente.

Alta: Ingreso o registro de una persona, producto u orden en el sistema.

Baja: Registro de la eliminación o desactivación de una persona, producto u orden dentro

del sistema.

PostgreSQL: Sistema de gestión de bases de datos relacional de código abierto, conocido por su robustez, extensibilidad y conformidad con el estándar SQL. Soporta características avanzadas como tipos de datos personalizados, replicación y manejo eficiente de grandes volúmenes de datos.

Interfaz: Medio a través del cual el usuario interactúa con el sistema, comprendiendo todos los puntos de contacto entre el usuario y la aplicación.

Node.js: Entorno de ejecución de JavaScript del lado del servidor, utilizado para desarrollar el backend de la plataforma.

React: Biblioteca de JavaScript para la construcción de interfaces de usuario dinámicas y responsivas, utilizada en el frontend del sistema.

Carrito de compras: Funcionalidad que permite al cliente seleccionar y acumular productos para su posterior compra dentro de la plataforma.

Orden de reparación: Registro generado cuando un cliente solicita el servicio técnico para su dispositivo móvil, que incluye la descripción del problema y el seguimiento del estado de la reparación.

2.5.2 Descripción general

2.5.2.1 Perspectiva del producto

El sistema "System NorTech" será una plataforma web accesible desde cualquier navegador, diseñada para operar tanto para el administrador de la tienda como para los clientes finales. Permitirá la gestión integral de productos, ventas, pedidos y servicios de reparación de dispositivos móviles, sin necesidad de instalación local.

2.5.2.2 Funcionalidad del producto

- Administración de productos.
- Administración de clientes y usuarios.
- Administración de ventas y pedidos.
- Gestión de solicitudes y seguimiento de reparaciones.
- Control de acceso y autenticación de usuarios.

2.5.2.3 Características de los usuarios

Tipo de usuario	Formación	Experiencia y especialización técnica
Administrador	Estudios terciarios completo	Experiencia en gestión comercial y administración de sistemas.
Cliente registrado	No requiere formación	Experiencia básica en navegación web y compras online
Cliente no registrado	No requiere formación	No requiere experiencia técnica previa.

Tabla Nro 15 - Características de los usuarios

2.5.2.4 Restricciones

- La plataforma funcionará como aplicación web, accesible desde cualquier navegador moderno.
- El sistema requerirá conexión a internet para su funcionamiento.
- El sistema no realizará gestión de tesorería y contabilidad.
- Lenguaje de programación utilizado: JavaScript.
- Tecnologías utilizadas:
 - Backend: Node.js (entorno de ejecución para JavaScript en el servidor).
 - Frontend: React (biblioteca para construir interfaces de usuario).
 - Base de datos: PostgreSQL, gestionado mediante supabase.
- El sistema permitirá la autenticación de usuarios.

2.5.2.5 Suposiciones y dependencias

- Cambios en la estrategia de negocio del cliente podrían requerir ajustes en el alcance del sistema.
- Podrán añadirse nuevas funcionalidades en futuras versiones del sistema.

2.5.3 Requisitos específicos

2.5.3.1 Requisitos funcionales

Nº	Descripción
RF1	El sistema debe permitir el alta de productos utilizando nombre, marca, precio

	y número de producto.
RF2	El sistema debe permitir modificar los datos de un producto utilizando su número de producto.
RF3	El sistema debe permitir dar de baja un producto utilizando su número de producto.
RF4	El sistema debe permitir el alta de un cliente utilizando correo electrónico y contraseña.
RF5	El sistema debe permitir modificar los datos de un cliente utilizando su correo electrónico y contraseña.
RF6	El sistema debe permitir dar de baja un cliente utilizando su correo electrónico y contraseña.
RF7	El sistema debe permitir al cliente que inició sesión realizar una compra.
RF8	El sistema debe permitir al administrador emitir un listado de las ventas realizadas.
RF9	El sistema debe permitir emitir un listado de los productos disponibles.
RF10	El sistema debe permitir emitir un listado de los clientes registrados.
RF11	El sistema debe permitir al usuario cerrar sesión.
RF12	El sistema debe permitir al cliente crear un carrito de compras.
RF13	El sistema debe permitir al cliente agregar o quitar productos del carrito.

RF14	El sistema debe permitir al administrador realizar compras en nombre de más de un cliente.
RF15	El sistema debe permitir al cliente registrar una solicitud de reparación indicando el modelo del dispositivo y la descripción del problema.
RF16	El sistema debe permitir al administrador actualizar el estado de una orden de reparación (recibido, en proceso, listo para retirar, entregado).
RF17	El sistema debe permitir al cliente consultar el estado actual de su orden de reparación.
RF18	El sistema debe permitir al administrador emitir un listado de todas las órdenes de reparación registradas.

Tabla Nro 16 - Requisitos funcionales

2.5.3.2 Requisitos no funcionales

N°	Descripción	Categoría
RF1	El sistema debe contar con una interfaz intuitiva y sencilla de utilizar.	Usabilidad
RF2	El sistema será implementado como una aplicación web accesible desde cualquier navegador moderno.	Portabilidad
RF3	El sistema de base de datos debe contar con respaldos	Eficiencia y rendimiento

	automáticos y periódicos.	
--	---------------------------	--

Tabla Nro 17 - Requisitos no funcionales

2.5.3.3 Requisitos organizacionales

N°	Descripción	Categoría
RF4	El sistema debe ser entregado antes del - de junio de 2026.	Entrega
RF5	El sistema deberá funcionar correctamente en cualquier navegador web actualizado.	Implementación

Tabla Nro 18 - Requisitos organizacionales

2.5.3.4 Requisitos externos

N°	Descripción	Categoría
RF6	El sistema deberá garantizar la seguridad de la información almacenada.	Seguridad
RF7	El sistema deberá garantizar que ninguna persona externa tenga acceso a los datos sensibles de clientes y transacciones.	Privacidad

Tabla Nro 19 - Requisitos externos

2.6 Descripción global

2.6.1 Perspectiva del producto

El sistema “System NorTech” será una aplicación web orientada a la gestión de ventas y reparaciones de dispositivos móviles. Permitirá a clientes y administradores interactuar con la plataforma mediante distintos niveles de acceso, centralizando la información de productos, usuarios, ventas y solicitudes de reparación.

2.6.2 Funcionalidad del producto

Administración de clientes.

Administración de usuarios.

Administración de productos.

El sistema permitirá visualizar productos, gestionar usuarios, realizar compras mediante un carrito de compras y registrar solicitudes de reparación. Además, contará con funcionalidades administrativas para la gestión de productos, ventas y control general de la plataforma.

2.6.3 Características del usuario

Tipo de usuario	Formación (nivel educativo)	Experiencia y Especialización técnica
Administrador	Estudios terciario completo	Experiencia en marketing y administración de empresas
Cliente	No requiere formación específica	Experiencia básica en navegación web y compras online.
Visitante	No requiere formación específica	No requiere experiencia técnica previa.

Tabla Nro 20 - Característica de los usuarios

2.6.4 Restricciones

- El sistema será desarrollado como aplicación web.
- El sistema requerirá conexión a internet para su funcionamiento.
- El desarrollo utilizará JavaScript como lenguaje principal.
- Se utilizarán React, Node.js y PostgreSQL como tecnologías base del sistema.

2.6.5 Suposiciones y dependencias

- El proyecto podrá ampliarse en futuras versiones incorporando nuevas funcionalidades.
- La disponibilidad de internet será necesaria para acceder a la plataforma.
- El desarrollo depende del correcto funcionamiento de los servicios utilizados, como Supabase y GitHub.

2.7 Plan de riesgos

2.7.1 Escala de evaluación

Probabilidad de ocurrencia	Rango
Muy baja	< 10%
Baja	10%-25%
Moderada	25%-50%
Alta	50%-75%
Muy alta	>75%

Tabla Nro 21 - Probabilidad de ocurrencia

Efecto
Insignificante
Tolerable
Serio
Catastrófico

Tabla Nro 22 - Niveles de efecto

Riesgo	Tipo	Probabilidad	Efecto
Cambio de requerimientos	Requerimientos	Moderada	Serio

durante el desarrollo			
Mala interpretación de los requisitos funcionales	Requerimientos	Moderada	Serio
Mala estimación del tiempo de desarrollo	Estimación	Alta	Serio
Falta de experiencia en las tecnologías del proyecto	Personal	Alta	Serio
Abandono o inactividad de un integrante del equipo	Personal	Baja	Catastrófico
Pérdida de avance del proyecto	Tecnología	Baja	Catastrófico
Necesidad de realizar más pruebas de las planificadas	Proyecto	Moderada	Serio
Interfaz poco amigable o difícil de comprender para el usuario	Producto	Baja	Serio
Las funcionalidades no cumplen con los requisitos definidos	Producto	Baja	Catastrófico

Tabla Nro 23 - Análisis de riesgo

Riesgo	Estrategía
Cambio de requerimientos durante el desarrollo	Documentar y acordar el alcance del sistema con el cliente desde el inicio.

Mala interpretación de los requisitos funcionales	Mantener comunicación continua. Documentar todos los requisitos de forma clara y validarlos antes de comenzar su implementación.
Mala estimación del tiempo de desarrollo	Dividir el desarrollo en iteraciones cortas con entregables parciales y revisar el cronograma periódicamente, ajustando prioridades según el avance real.
Falta de experiencia en las tecnologías del proyecto	Destinar tiempo previo al desarrollo para capacitación mediante documentación oficial, tutoriales y proyectos de práctica en React, Node.js y PostgreSQL.
Abandono o inactividad de un integrante del equipo	Mantener documentación actualizada del avance y del código, de modo que cualquiera de los dos integrantes pueda retomar el trabajo del otro con el menor impacto posible.
Pérdida de avance del proyecto	Utilizar un repositorio remoto en GitHub con commits frecuentes.
Necesidad de realizar más pruebas de las planificadas	Incorporar instancias de prueba desde las etapas tempranas del desarrollo, evitando concentrarse únicamente al final del ciclo.
Interfaz poco amigable o difícil de comprender	Realizar revisiones de usabilidad durante el desarrollo, tomando como referencia buenas prácticas de diseño de interfaces.
Las funcionalidades no cumplen con los requisitos	Validar cada módulo desarrollado contra los requisitos documentados antes de avanzar al siguiente, garantizando trazabilidad entre implementación y especificación.

Tabla Nro 24 - Planificación de riesgos

2.8 Modelado del sistema

2.8.1 Diagrama de casos de uso

Caso de uso para el visitante

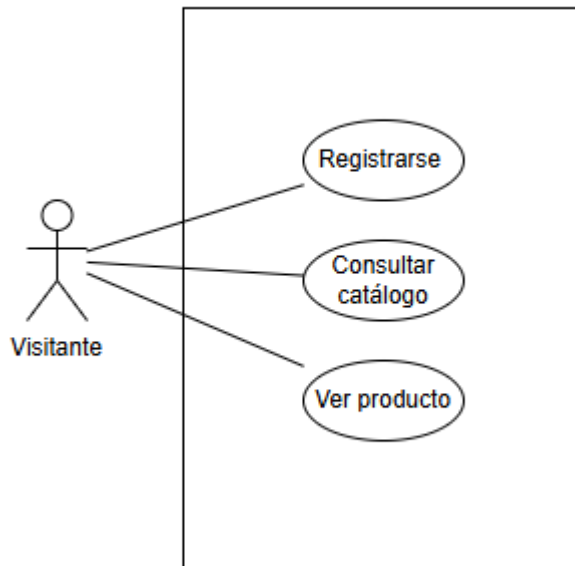


Figura Nro 4 - Caso de uso: visitante

Caso de uso para el cliente

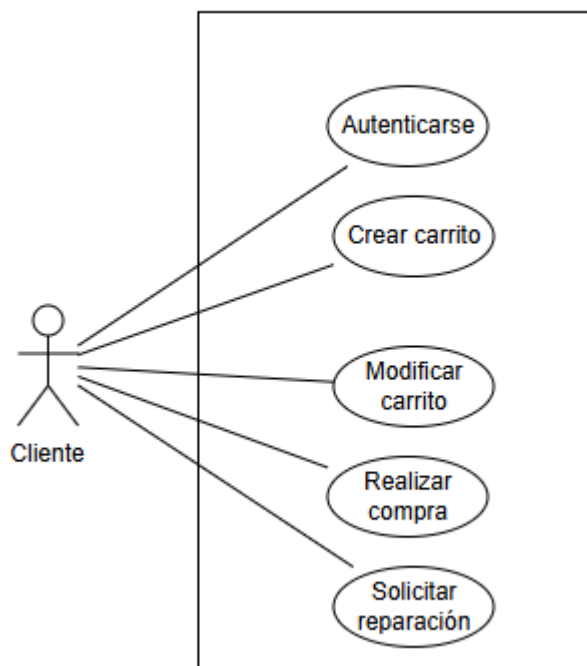


Figura Nro 5 - Caso de uso: cliente

Caso de uso para el administrador

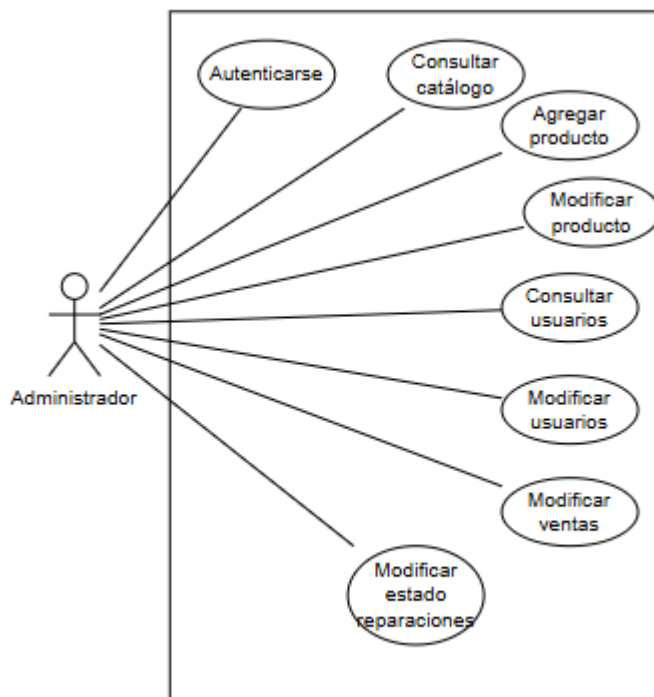


Figura Nro 6 - Caso de uso: administrador

2.8.2 Conversaciones

Conversación 1

Caso de uso: Realizar compra

Actor: Cliente (c)

Acción	Curso Normal	Curso Alternativo
1. C: presiona la pestaña "catálogo"	1.1. S: muestra la pestaña catálogo. mostrarProductos()	
2.C:presiona el botón	2.1. S: Añade al carrito de	

"agregar al carrito"	compras el producto. agregarAlCarrito(id_producto) 2.1.2 S: muestra un mensaje: "el producto fue agregado al carrito". mostrarMensaje()	
3.C: presiona la pestaña "Carrito"	3.1. S: muestra la pestaña "Carrito"	
4.C: Presiona el botón "Finalizar compra".		
5. S: Solicita confirmación de pago. solicitarPago(id_cliente, prod, monto)	5.1 C: confirma pago	5.2 C: no confirma 5.2.1 S: muestra un mensaje: "compra cancelada". mostrarMensaje()
6. S: Verifica el pago	6.1 Pago realizado 6.1.2 S: Registra la venta crearVenta(id_cliente, prod, monto) 6.1.3 S: Muestra el mensaje: "Venta creada exitosamente" mostrarMensaje()	6.2 El pago presento una falla 6.2.1 S: Muestra el mensaje: "Se ha producido un error. Reintentar" mostrarMensaje() 6.2.2 Ir al paso 4.
7. S: Actualiza el stock actualizarProducto(id, cant)		
8. Fin del caso de uso		

Tabla Nro 25 - Conversación "Agregar producto".

Conversación 2

Caso de uso: Agregar producto

Actor: Administrador(a)

Acción	Curso Normal	Curso Alternativo
1.A: presiona “agregar nuevo producto”		
2. S: lista las categorías getCategorias()		
3. S: lista las marcas getMarcas()		
3. A: ingresa nombre, imagen, marca, descripción, precio costo, precio venta, stock, stock mínimo y categoría del producto		
4. A: presiona “guardar producto”		
5. S: verifica datos correctos	5.1 datos correctos 5.1.1 S: agrega producto crearProducto(nombre, imagen, marca, descripcion, precio_costo, precio_venta, stock, stock_min, idcategoria) 5.1.2 S: muestra el mensaje: “Producto agregado con éxito”	5.2 datos incorrectos 5.2.1 S: muestra el mensaje: “Los datos son incorrectos” 5.2.2 Ir al paso 3
6. A: presiona el botón “aceptar”		
7. Fin del caso de uso		

Tabla Nro 26 - Conversación “Agregar producto”.

Conversación 3

Caso de uso: Solicitar reparación

Actor: Cliente

Acción	Curso Normal	Curso Alternativo
1. c: presiona la pestaña "reparación"		
2. s: muestra el formulario formReparacion()		
3. c:rellena el formulario		
4. c: presiona el botón "Solicitar"		
5. s: valida los datos ingresados	5.1 Campos completos 5.1.1 s: registra la solicitud registrarReparacion(id_cliente, modelo, descripcion) 5.1.2 s: muestra el mensaje: "Solicitud registrada con éxito" mostrarMensaje()	5.2 Campos incompletos 5.2.1 s: muestra el mensaje: "Debe completar todos los campos" mostrarMensaje() 5.2.2 Ir al paso 3
6. Fin del caso de uso		

Tabla Nro 27 - Conversación "Solicitar reparación".

2.8.3 Diagramas de secuencia

Diagrama de secuencia número 1

Caso de uso: Realizar compra

Actor: Cliente

Curso Normal:

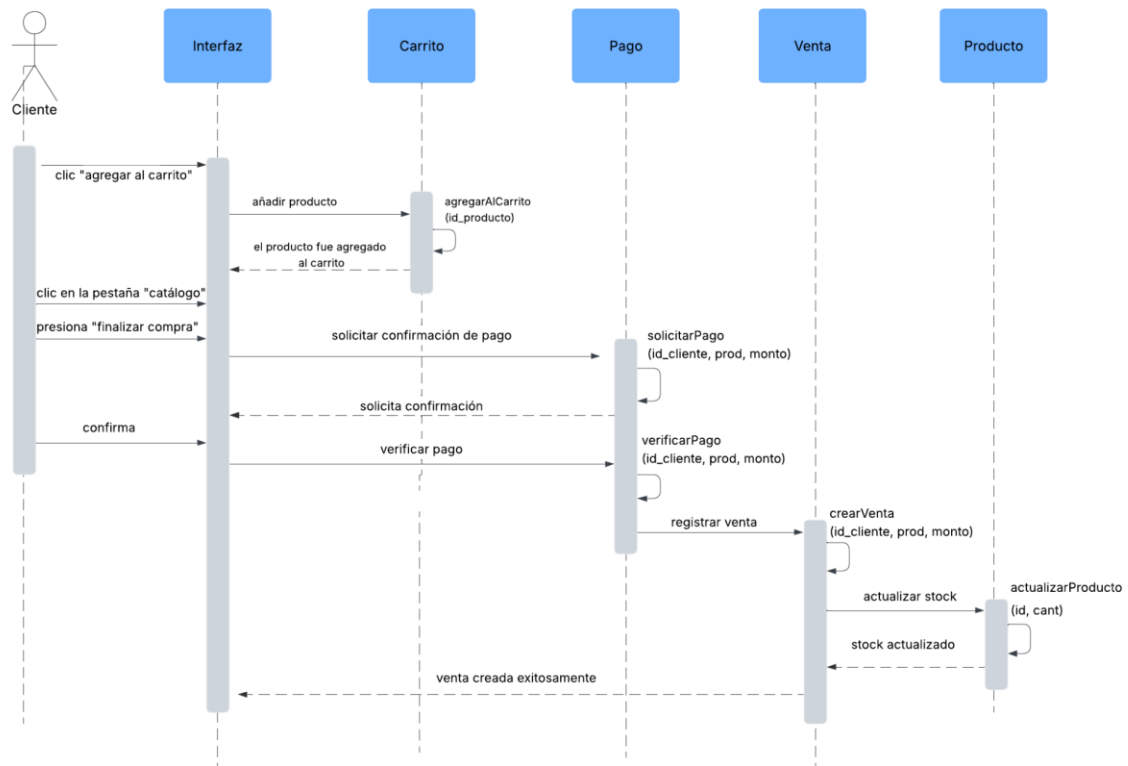


Figura Nro 7 - Diagrama de secuencia:realizar compra.

Este diagrama representa la interacción entre el cliente, la interfaz del sistema y la lógica de negocio durante el proceso de compra. Se contemplan operaciones como la validación del carrito, el procesamiento del pago, el registro de la venta y la actualización del stock.

Diagrama de secuencia número 2

Caso de uso: Agregar producto

Actor: Administrador

Curso normal:

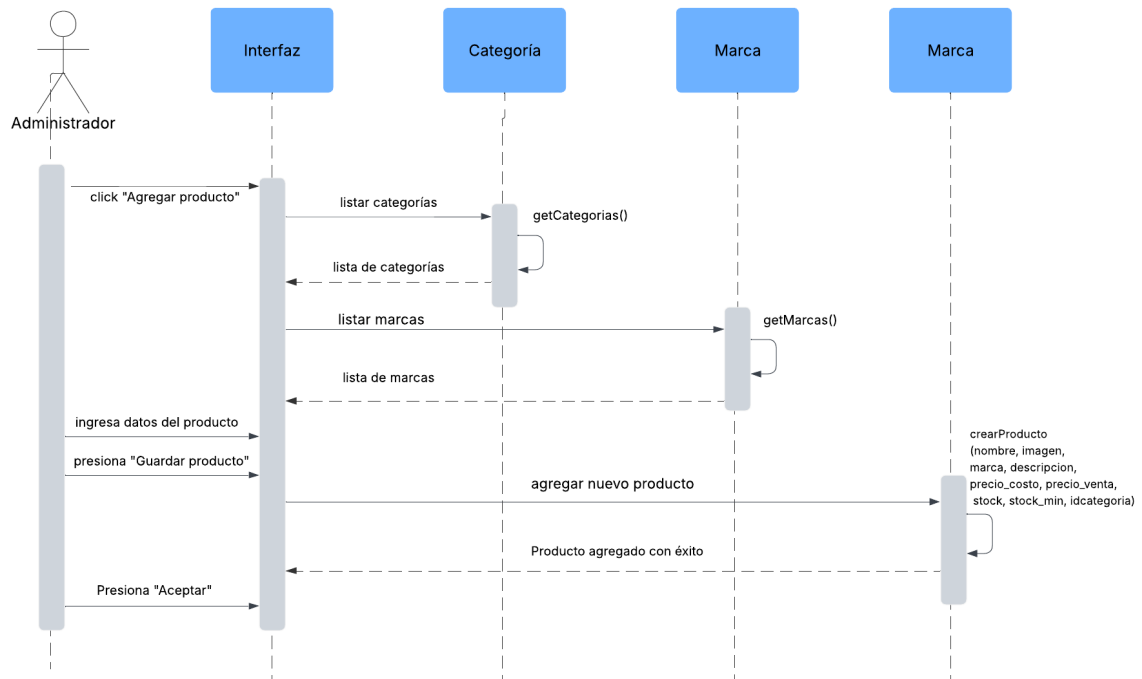


Figura Nro 8 - Diagrama de secuencia: agregar producto

Este diagrama modela el flujo utilizado por el administrador para registrar nuevos productos en el sistema, incluyendo la validación de datos y el almacenamiento de la información en la base de datos.

Diagrama de secuencia número 3

Caso de uso: Solicitar reparación

Actor: Cliente

Curso Normal:

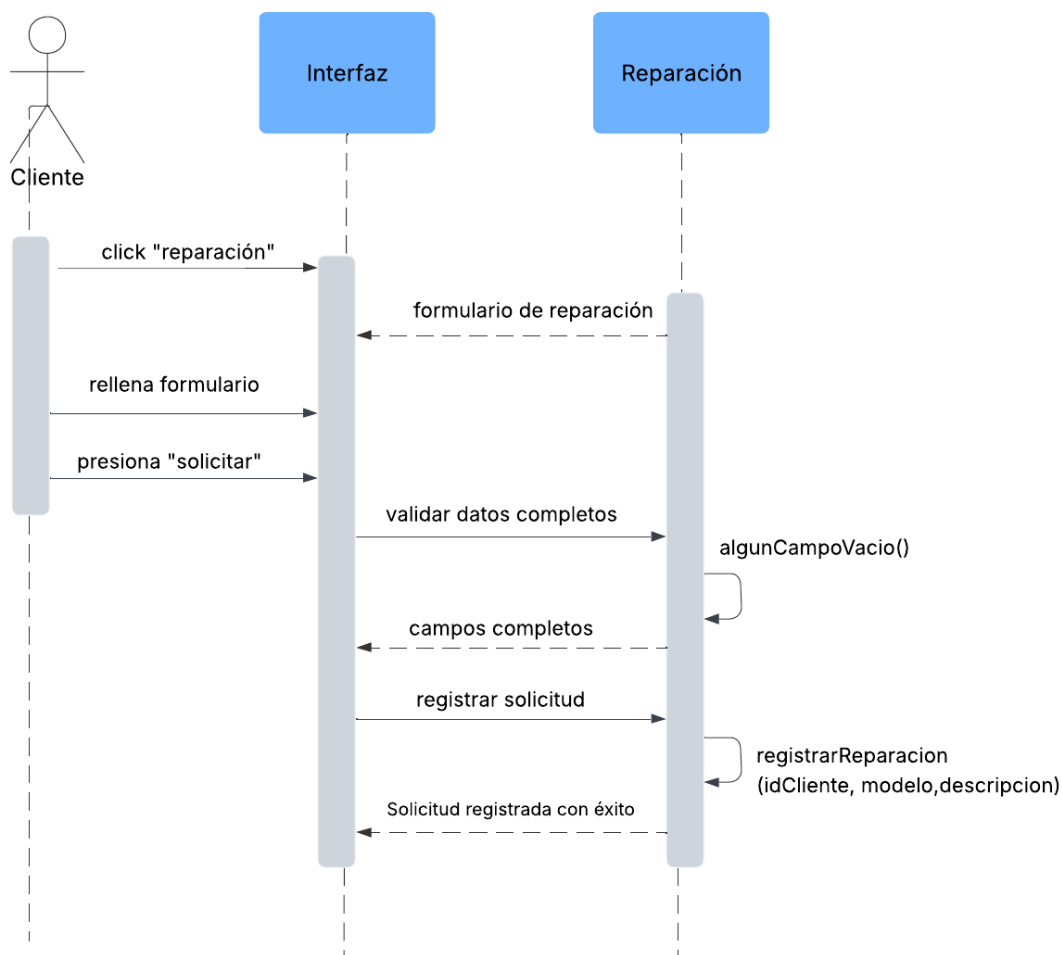


Figura Nro 9 - Diagrama de secuencia:solicitar reparación.

Este diagrama representa el proceso de registro de solicitudes de reparación realizadas por los clientes, contemplando la validación de los datos ingresados y el almacenamiento de la orden en el sistema.

2.8.4 Contrato de operaciones críticas

Nombre: realizarCompra (id_cliente, detalle_compra)

Referencias cruzadas: Casos de uso: realizar compra

Responsabilidades: El sistema debe verificar que el cliente se encuentre registrado y que el carrito contenga al menos un producto. Luego debe procesar el pago, registrar la venta, actualizar el stock de cada producto vendido y generar una confirmación de compra para el cliente.

Excepciones:

- Si el cliente no inició sesión, el sistema lo redirige a la pantalla de autenticación.
- Si uno o más productos del carrito no tienen stock suficiente, el sistema notifica al cliente e impide continuar con la compra hasta que se corrija el carrito.
- Si el método de pago es rechazado, el sistema informa el error y permite al cliente reintentar.
- Si ocurre un error de conexión durante el proceso, el sistema cancela la transacción y no registra ningún cambio.

Precondiciones:

- El cliente debe estar registrado en el sistema.
- El carrito debe contener al menos un producto.
- Todos los productos del carrito deben tener stock disponible.

Postcondiciones:

- La venta queda registrada en el sistema con fecha, detalle de productos, monto total e identificación del cliente.
- Se actualiza el stock del sistema de cada producto vendido.
- El carrito del cliente es vaciado una vez confirmada la compra.
- El cliente recibe una confirmación de la compra realizada.

Tabla Nro 28 - Contrato de operación 1

Nombre: solicitarReparacion(id_cliente, modelo, descripcion)

Referencias cruzadas: Caso de uso: solicitar reparación

Responsabilidades: El sistema debe verificar que el cliente se encuentre registrado y que los datos del dispositivo y la descripción del problema hayan sido completados correctamente. Luego debe registrar la solicitud de reparación, asignarle el estado inicial "Recibido" y dejarla disponible tanto para consulta del cliente como para gestión por parte del administrador.

Excepciones:

- Si el cliente no inició sesión, el sistema lo redirige a la pantalla de autenticación.
- Si los campos obligatorios del formulario se encuentran incompletos, el sistema notifica al cliente que debe completar los datos para continuar.
- Si ocurre un error al registrar la orden, el sistema informa el problema y no genera

ningún registro.

Precondiciones:

- El cliente debe estar registrado en el sistema.
- El cliente debe haber ingresado el modelo del dispositivo y una descripción del problema.
- Todos los campos obligatorios del formulario de reparación deben estar completos.

Postcondiciones:

- La orden de reparación queda registrada en el sistema con un identificador único, la fecha de solicitud, el modelo del dispositivo, la descripción del problema y la identificación del cliente.
- La orden queda asociada al estado inicial "Recibido".
- El cliente puede consultar el estado de su reparación desde su perfil.
- El administrador puede visualizar y gestionar la nueva orden desde el panel de administración.

Tabla Nro 29 - Contrato de operación 2

2.8.5 Diagrama de entidad-relación

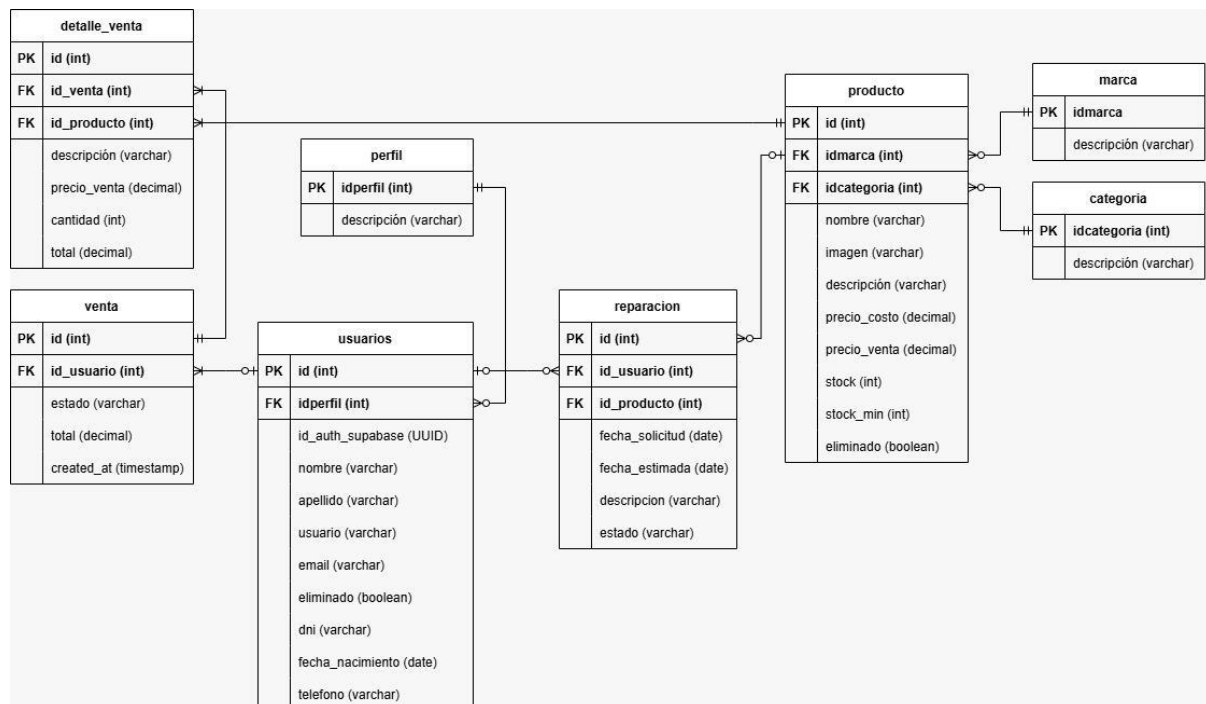


Figura Nro 10 - Diagrama entidad-relación del sistema Nortech

Descripción del modelo de datos

En este se representan las principales entidades y relaciones necesarias para el

funcionamiento del sistema NorTech. Entre las entidades más importantes se encuentran usuarios, productos, ventas, detalles de venta y solicitudes de reparación.

La entidad usuarios almacena la información de autenticación y los distintos roles del sistema, permitiendo diferenciar administradores y clientes. La entidad productos contiene los datos relacionados con los dispositivos publicados en el catálogo, incluyendo nombre, descripción, precio y el stock disponible.

La entidad ventas registra las compras realizadas por los clientes, mientras que `detalle_ventas` permite almacenar los productos asociados a cada venta junto con sus cantidades correspondientes. Esto permite mantener una estructura organizada y escalable para futuras ampliaciones del sistema.

Finalmente, el módulo de reparaciones incorpora entidades relacionadas con las solicitudes realizadas por los clientes, facilitando el seguimiento y control de los servicios técnicos ofrecidos por la plataforma.

3. Herramientas y/o lenguajes de programación

Las herramientas utilizadas en el desarrollo de este proyecto son:

Microsoft Word Online es la versión basada en la nube del procesador de texto Microsoft Word, accesible desde cualquier navegador web como parte de la suite Microsoft 365. Fue empleada para la redacción y edición colaborativa de la documentación del proyecto.

Google Meet es una plataforma de videoconferencia desarrollada por Google, accesible desde cualquier navegador web o dispositivo móvil sin necesidad de instalación. Fue utilizada para llevar a cabo las reuniones virtuales del equipo de desarrollo, facilitando la comunicación, el debate de ideas y la toma de decisiones a lo largo de las distintas etapas del proyecto.

Visual Studio Code fue utilizado como editor principal para el desarrollo frontend y backend del sistema ecommerce de celulares y el módulo de reparaciones debido a su integración con extensiones para React, Node.js y control de versiones con GitHub. Su amplio ecosistema de extensiones lo convierte en una herramienta muy versátil para el desarrollo web.

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto, mantenido por una comunidad global de desarrolladores. Reconocido por su conformidad con el estándar ANSI SQL y por sus características avanzadas como tipos de datos

personalizados, replicación y control de concurrencia multiversión (MVCC). Permite interactuar mediante SQL estándar y a través de su lenguaje procedural PL/pgSQL, posibilitando la definición de funciones, procedimientos almacenados y triggers para gestionar la lógica de datos de productos, clientes, pedidos y órdenes de reparación.

Supabase es una plataforma de Database as a Service (DBaaS) que permite gestionar instancias de PostgreSQL en la nube de forma escalable y segura. Facilita la administración, el monitoreo y los respaldos automáticos de la base de datos, garantizando alta disponibilidad y rendimiento para la tienda online.

Trello es una herramienta de gestión de proyectos con interfaz web y aplicaciones móviles para iOS y Android. Fue utilizada para organizar las tareas del equipo, hacer seguimiento del progreso y coordinar las distintas etapas del desarrollo del sistema.

Visual Paradigm es una herramienta de modelado UML que ofrece soporte completo para la creación de distintos tipos de diagramas bajo el estándar UML, incluyendo diagramas de casos de uso, clases, componentes y actividades. Fue utilizada para elaborar los diagramas de casos de uso del proyecto, representando visualmente las interacciones entre los actores —clientes, administradores— y el sistema.

LucidChart es una herramienta de diagramación colaborativa basada en la nube que permite crear y compartir diagramas de manera ágil desde cualquier navegador. Fue utilizada para la elaboración de los diagramas de secuencia del proyecto, modelando la interacción entre los objetos del sistema en cada uno de los casos de uso desarrollados.

Node.js es un entorno de ejecución de JavaScript del lado del servidor. Fue utilizado para desarrollar el backend de la plataforma, implementando la lógica de negocio, el manejo de datos de productos, usuarios, pedidos de compra y órdenes de reparación, así como la conexión con la base de datos PostgreSQL.

React es una biblioteca de JavaScript para la construcción de interfaces de usuario. Fue utilizada para desarrollar el frontend de la tienda online, creando una interfaz dinámica, modular y responsiva que permite a los clientes explorar el catálogo de celulares, registrarse, iniciar sesión, realizar compras y gestionar solicitudes de reparación de sus dispositivos.

Vite es una herramienta de desarrollo para proyectos frontend que permite crear aplicaciones web de manera rápida y eficiente. Fue utilizada como entorno de desarrollo para el frontend en React, facilitando la ejecución del proyecto, la recarga en tiempo real y

la optimización del rendimiento durante el desarrollo.

4. Resultados

1. Visualización de los productos cargados en el catálogo.

- Como visitante:

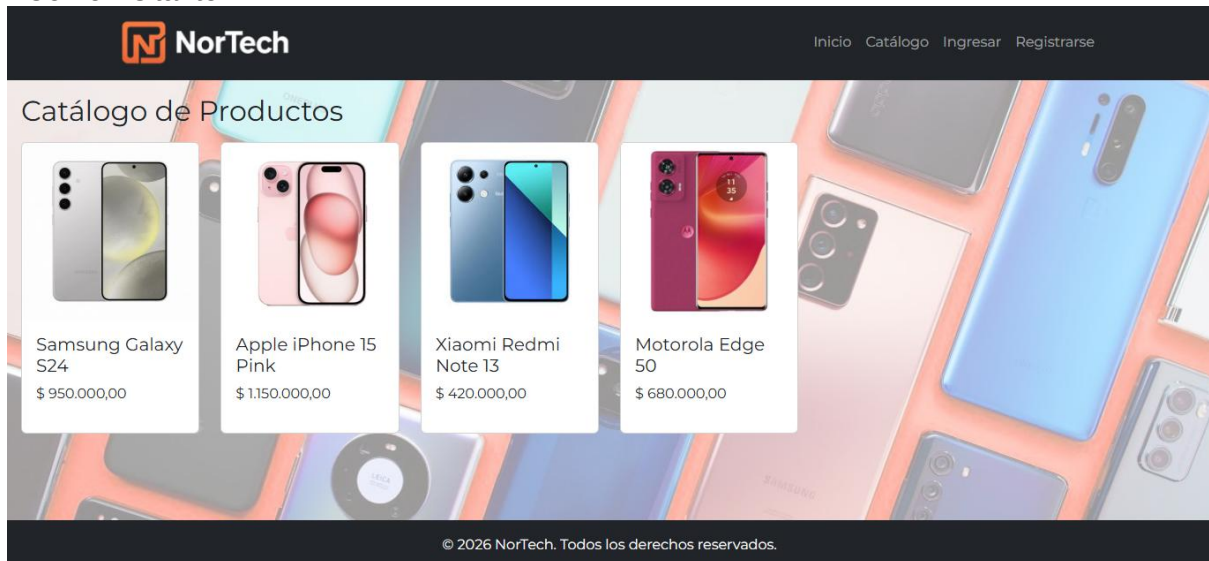


Figura Nro 11 - Catálogo de productos (como visitante)

- Como cliente:

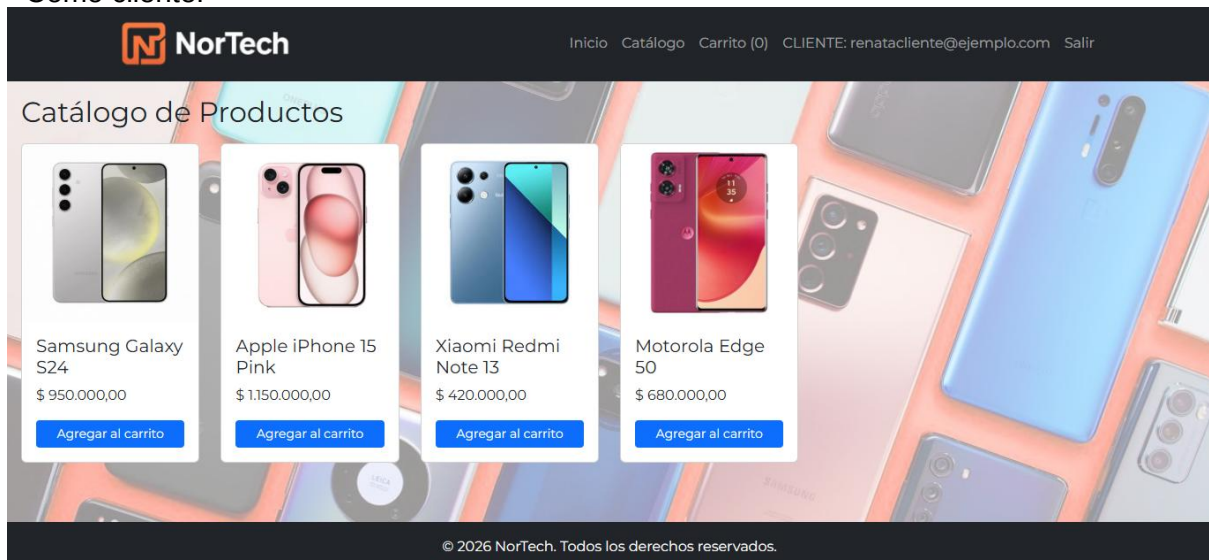


Figura Nro 12 - Catálogo de productos (como cliente)

- Como administrador:

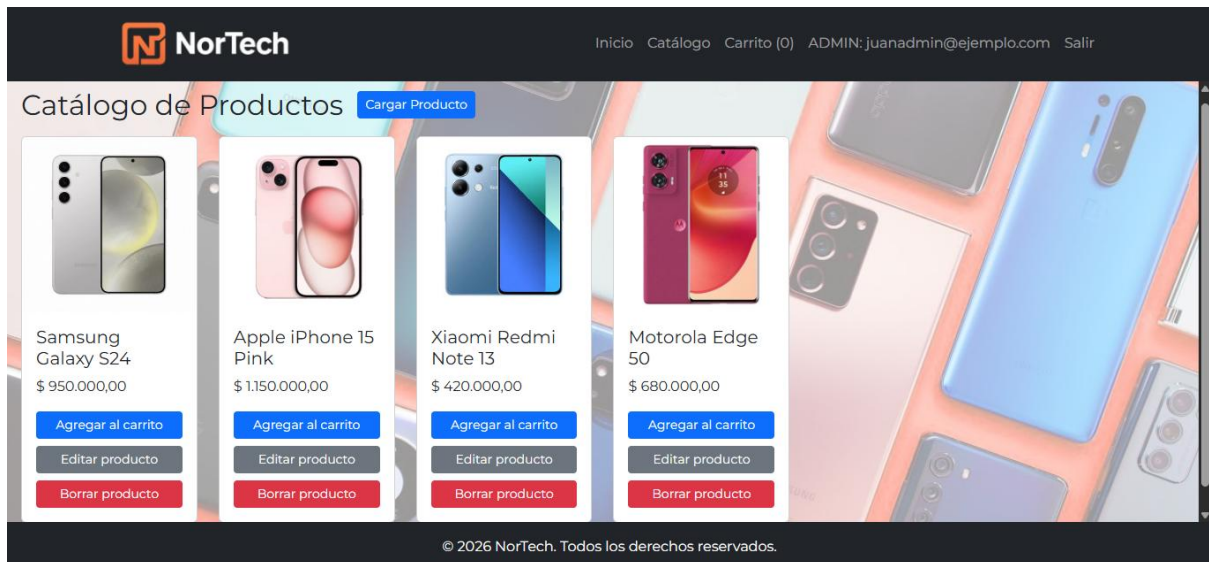
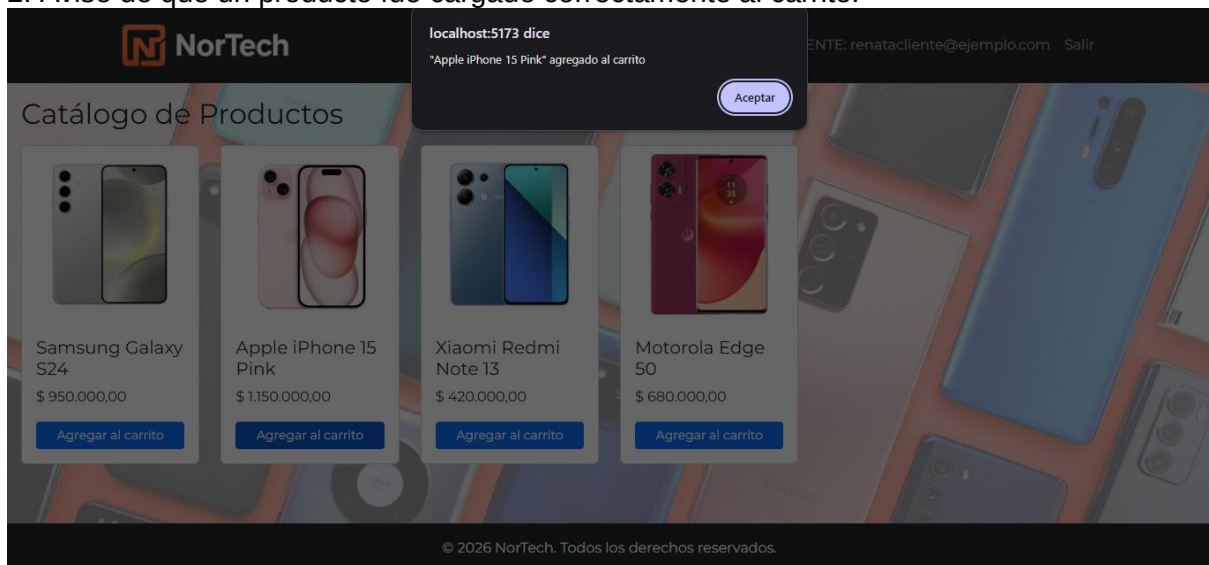


Figura Nro 13 - Catálogo de productos (como administrador)

2. Aviso de que un producto fue cargado correctamente al carrito.



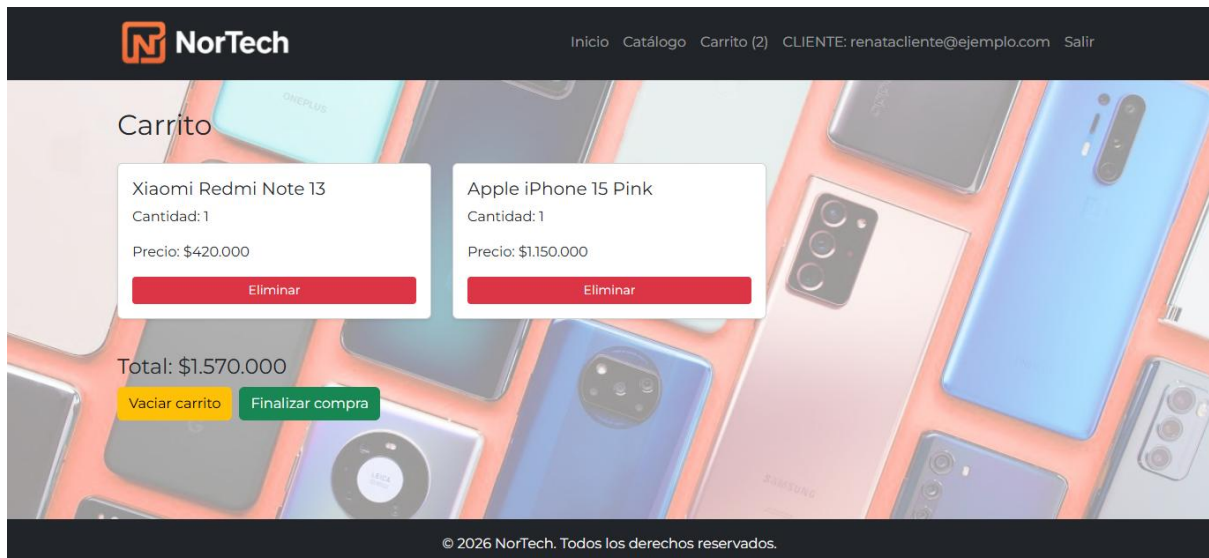


Figura Nro 15 - Carrito cargado con productos

4. Aviso de que la compra fue realizada con éxito.

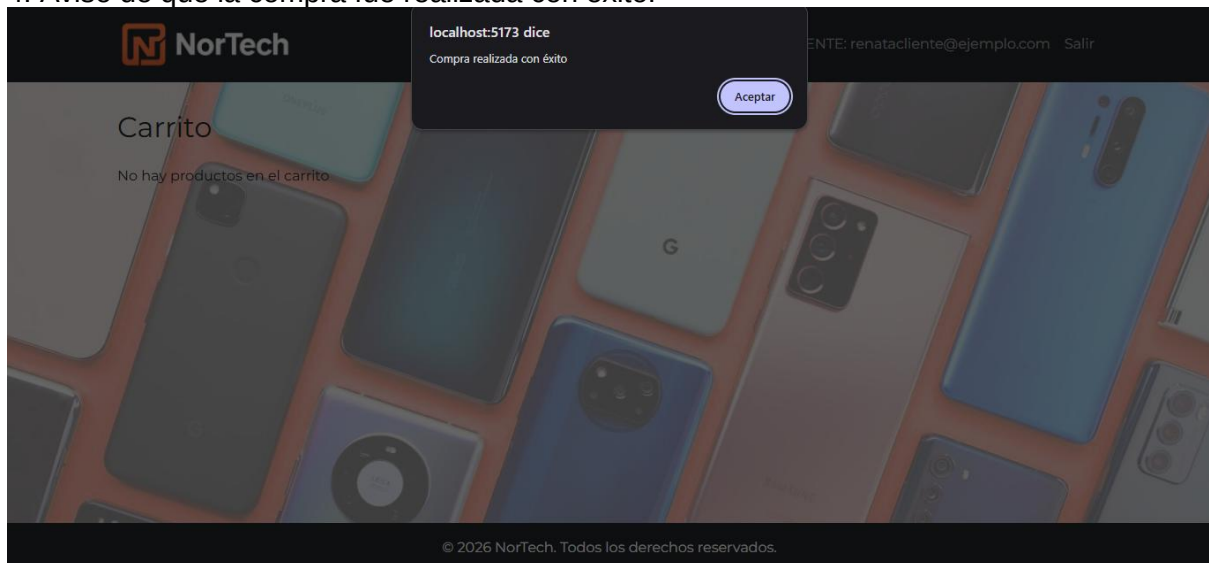


Figura Nro 16 - Compra finalizada

La funcionalidad implementada "Carrito de compras" corresponde a:

- RF12: Crear carrito
- RF13: Agregar/Quitar productos
- Caso de uso: Cargar carrito
- Diagrama de secuencia: Figura Nro 7
- Contrato: realizarCompra

4.1 Estado actual del módulo de reparaciones

El módulo de reparaciones fue trabajado principalmente a nivel de análisis y diseño, por lo que su estructura quedó representada en el DER, requisitos funcionales, conversaciones,

diagramas de secuencia y contratos de operación correspondientes. La implementación funcional de este módulo quedó planificada para la segunda entrega del proyecto.

5. Conclusión

En el lapso de esta primera entrega se logró desarrollar y documentar las principales bases del sistema NorTech, incluyendo el análisis de requisitos, modelado UML, planificación del proyecto y la implementación funcional del carrito de compras dentro de la plataforma ecommerce.

Además, el proyecto permitió aplicar conceptos vistos durante la materia relacionados con arquitectura de software, metodologías ágiles, modelado de datos y gestión de proyectos. Por otro lado, quedó definida y modelada la segunda funcionalidad (módulo de reparaciones) cuya implementación será abordada en la siguiente etapa del trabajo.

5.1 Bibliografía

- Mill, H. D. (1980). *Software Productivity*. Little, Brown and Company.
- Pressman, R. S. (2010). *Ingeniería de Software: Un enfoque práctico* (7.^a ed.). McGraw-Hill.
- Pressman, R. S., & Maxim, B. R. (2015). *Software Engineering: A Practitioner's Approach* (8.^a ed.). McGraw-Hill Education.
- Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*.
- Sommerville, I. (2011). *Ingeniería del Software* (9.^a ed.). Pearson Educación.